

v0.3.4 Serenity Skill Memo 生成

Codex 分批执行计划书。本文只定义开发、模型接入、本地发布、GitHub 推送和 Vercel 部署流程，不直接执行产品开发。

版本	v0.3.4
定位	参考朋友页面的结构化效果，把 Serenity Skills 从数据罗列升级为证据驱动的框架化 memo。
预计工作量	2-3 个 Codex 工作轮次，重点是 schema、prompt 和 UI 模块映射。
建议分支	codex/v0.3.4-serenity-skill-memo
完成标志	Serenity Alpha、Bayesian、GF-DMA、TAM-Adj-PEG、研究备忘录都能稳定展示结构化分析。

总体链路

多数据源抓取 -> 清洗成 evidence / facts -> 结构化数据类型 -> 喂给大模型 -> 生成模块化分析内容 -> 前端渲染成网页内容。

计划书中的输出均要求证据可追溯、缺失可解释、模型可关闭、部署可回滚。

版本目标

- 把 Serenity Skills 改为由 facts/evidence + LLM 共同生成的结构化 memo，而不是展示空白或错误数据。
- 每个 Skill 有固定输出骨架：核心观察、证据、变量、分歧点、后续观察项。
- 保留确定性计算结果，例如估值倍数、增长率、健康指数，LLM 只负责归纳和解释。
- 所有模块输出必须带 evidenceRefs 和 calculationRefs。
- 如果某模块数据不足，展示缺失原因和下一步需要的数据，而不是空白卡片。

主要改动文件

- 新增 src/lib/serenity/llmSerenityMemo.ts：统一 Serenity memo 生成入口。
- 新增 src/lib/llm/prompts/serenitySkills.ts：各 skill 的 prompt 模板。
- 新增 src/lib/llm/schemas/serenityMemo.ts：SkillMemo、MemoSection、EvidenceLink schema。
- 更新 src/lib/serenity/index.ts：把 real-data bundle 和 LLM memo 合并输出。
- 更新 src/components/serenity/*Panel.tsx：按 memo schema 渲染。
- 更新 GeneratedCardPreview 中 Serenity 区块：标签、空态、证据列表和免责声明统一。

详细开发步骤

- 先抓当前 Serenity 展示错误：记录哪些字段为空、哪些字段类型不匹配、哪些模块只接到 mock fallback。
- 定义 SerenityMemoSchema：overview、skillCards、crossSkillTensions、watchItems、dataLimitations。
- 为 Serenity Alpha 设计输入：新闻事件如果未接入，就仅使用财报、指引、价格和估值 facts，不生成新闻结论。
- 为 Bayesian 设计输入：基准假设、上调/下调变量、证据权重，保留计算过程。
- 为 GF-DMA 设计输入：增长、盈利质量、资产负债、价格波动、指引变化等可量化 facts。
- 为 TAM-Adj-PEG 设计输入：增长率、估值倍数、市场空间证据；缺少 TAM 时必须标记为数据不足。
- 将 LLM 输出渲染成固定 UI，不直接渲染模型长文本。

DeepSeek / 大模型接入位置

- 复用 v0.3.3 的 LLM provider，不再新增一套调用逻辑。
- Serenity prompt 必须比 Research Brief 更严格：每个结论都需要 evidenceRefs。
- 模型参数建议使用 deepseek-v4-pro；若响应时间影响体验，后台生成并缓存结果。
- 对输出做长度限制，避免 Serenity 面板变成长篇报告。

验收标准

- Serenity Skills 不再显示全部错误或无法查询。
- 至少四个 Skill tab 有结构化内容或清晰缺失态。
- 研究备忘录不再显示暂无数据，除非 facts/evidence 确实为空。
- 每个 Skill 至少有一条证据来源或一条明确缺失原因。
- npm.cmd run lint 和 npm.cmd run build 通过。

Codex 首轮执行 Prompt

请继续 v0.3.4: 按本 PDF 实现 Serenity Skill Memo 生成, 复用 v0.3.3 LLM provider 和 v0.3.2 facts/evidence。目标是让 Serenity Skills 以固定结构展示分析, 而不是空白或错误数据。完成后本地验收并准备推送。

通用执行前检查

- 进入项目目录并确认当前分支、远端、工作区差异。不要覆盖用户未提交的改动; 若已有改动与本版本相关, 先阅读再合并。
- 先阅读 CLAUDE.md、README.md、docs/REAL_DATA_INTEGRATION_PLAN.md, 以及本版本涉及的 src/lib、src/components、src/types 文件。
- 每次修改完成后优先运行 npm.cmd run lint 和 npm.cmd run build; Windows PowerShell 下优先使用 npm.cmd。
- 所有外部数据源失败时必须返回可解释的缺失状态, 不能让页面直接 500。
- 研究内容必须带 evidenceRef 或 sourceUrl; 没有证据时只能显示缺失原因, 不能补写事实。

本地发布流程

每个小版本完成后都要先在本地跑通, 再决定是否推送。

```
cd "C:\Users\■■■■■■\Desktop\moki-research-card"
npm.cmd install
npm.cmd run lint
npm.cmd run build
npm.cmd run dev -- --hostname 127.0.0.1 --port 3000
# ■■■■■■ http://localhost:3000/ ■■■■■■
```

GitHub 推送流程

建议每个小版本单独分支和单独 commit, 用户确认后再进入 main。

```
git status -sb
git checkout -b codex/v0.3.4-short-name
git add -A
git commit -m "feat: v0.3.4 short release title"
git push origin codex/v0.3.4-short-name

# ■■■■■■■ main ■■■
git checkout main
git pull --ff-only origin main
git merge --ff-only codex/v0.3.4-short-name
git push origin main
```

Vercel 部署流程

- 如果 Vercel 项目已连接 GitHub, 推送到 main 后会触发生产部署; 预览分支会生成 Preview Deployment。
- 在 Vercel Project Settings - Environment Variables 中配置本版本需要的变量, 并同时勾选 Production、Preview、Development。
- 部署后检查 Vercel Build Logs、Function Logs、页面首屏、NVDA 样例、无数据 ticker 样例和移动端布局。
- 生产环境不要暴露任何 API Key 到 NEXT_PUBLIC_ 变量; 所有抓取和 LLM 请求必须在服务端执行。

通用回滚流程

- 若本地 lint 或 build 失败，先修复，不进入 GitHub 推送。
- 若 Vercel 部署失败，保留失败部署日志，回退到上一个成功 main commit，或重新推送修复 commit。
- 若外部数据源临时失败，页面必须保留上一版可用的缺失态和数据源说明。
- 若 LLM 输出未通过 schema 校验，回退到确定性摘要，不渲染未经校验的文本。

参考链接

- SEC EDGAR APIs: <https://www.sec.gov/search-filings/edgar-application-programming-interfaces>
- DeepSeek API Docs: <https://api-docs.deepseek.com/>
- Vercel Deployments: <https://vercel.com/docs/deployments>
- Vercel Environment Variables: <https://vercel.com/docs/environment-variables>